

PSS06

6-Member Development Plan

Roles • Responsibilities • 3-Layer Architecture • Integration

Goal: six clear owners, one integrated system, no duplicated work.

1. The 3 Layers

Layer	What it does	Main work
Layer 1 — Data & Search	Handles the registered newspaper-title database and finds likely conflicts.	Data cleaning • PostgreSQL • pgvector • embeddings • indexing • candidate retrieval
Layer 2 — AI/ML & Verification	Checks how similar a new title is and applies the defined rules.	Normalization • language detection • transliteration • lexical/phonetic/semantic similarity • core-word matching • rule engine • scoring • RAG explanation
Layer 3 — Application & Agent	Lets users interact with the system and optionally generates better title suggestions.	FastAPI • Next.js • results/dashboard • Generator • Verifier • Ranker

2. Division of Work Among 6 Members

Member	Layer	Role	Main Responsibility
1	Layer 3	Backend + System Integration	Build APIs and connect every module into one verification pipeline.
2	Layer 2	NLP / ML Engineer	Build preprocessing and all major similarity calculations.
3	Layer 1	Database + Search Engineer	Manage the title dataset, database, embeddings and fast candidate retrieval.
4	Layer 2	Rules + RAG + Explainability	Convert official rules into checks and explain results using relevant guidelines.
5	Layer 3	Title Generation / Agentic Workflow	Generate alternative titles, verify them and rank the best ones.
6	Layer 3	Frontend + Dashboard + QA	Build the user interface, visualize results and own end-to-end testing.

Member 1 — Backend + System Integration

Layer: Layer 3

- Build the FastAPI backend.
- Define request/response formats for every module.
- Create the main /verify-title pipeline.
- Connect search, ML, rules and explanation modules.
- Handle errors, logging and common backend utilities.
- Keep the integrated application stable.

Suggested technologies: Python • FastAPI • Pydantic • PostgreSQL • Docker

Member 2 — NLP / ML Engineer

Layer: Layer 2

- Clean and normalize submitted titles.
- Detect language and handle transliteration.
- Implement lexical similarity.
- Implement phonetic similarity.
- Implement semantic similarity using embeddings.
- Implement core-word matching.
- Return separate scores so the backend can combine them.

Suggested technologies: Python • Sentence Transformers • RapidFuzz • scikit-learn • NumPy

Member 3 — Database + Search Engineer

Layer: Layer 1

- Clean and import the registered-title dataset.
- Design PostgreSQL tables.
- Store title embeddings.
- Set up pgvector.
- Create indexes for fast retrieval.
- Build top-K candidate search.
- Optimize database/search performance.

Suggested technologies: PostgreSQL • pgvector • Python • Pandas • SQL

Member 4 — Rules + RAG + Explainability

Layer: Layer 2

- Study the official rules relevant to title verification.
- Convert supported rules into deterministic checks.
- Create rule IDs, severity levels and messages.
- Build the guideline/document retrieval system.
- Retrieve the relevant guideline when a rule is triggered.
- Generate clear, grounded explanations.
- Test rule correctness and prevent unsupported claims.

Suggested technologies: Python • Rule Engine • RAG • Embeddings • pgvector • LLM API

Member 5 — Title Generation / Agentic Workflow

Layer: Layer 3

- Build the title-generation workflow.
- Generate alternative newspaper titles from user requirements.
- Send generated titles to the real verification engine.
- Discard titles that fail verification.
- Rank the surviving alternatives.
- Implement retries/feedback when generated titles are poor.

Suggested technologies: Python • LangGraph • LLM API • FastAPI tools

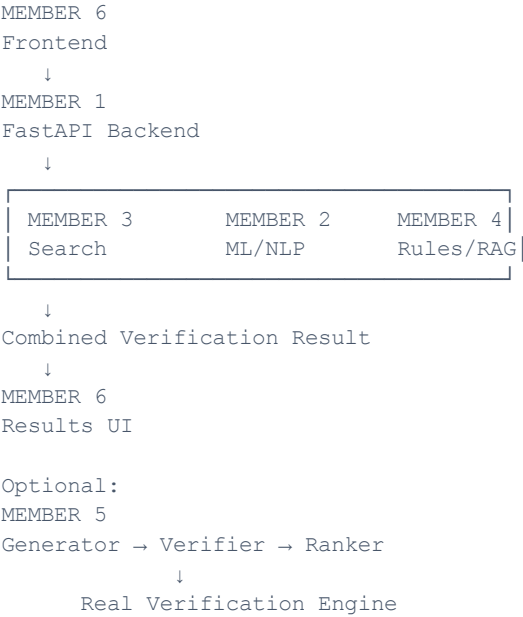
Member 6 — Frontend + Dashboard + QA

Layer: Layer 3

- Build the title submission interface.
- Display verification results clearly.
- Show conflicting registered titles.
- Visualize similarity scores and rule violations.
- Show explanations and suggested alternatives.
- Build a basic dashboard if time permits.
- Write end-to-end tests and polish the final demo.

Suggested technologies: Next.js • TypeScript • Tailwind • Recharts • Playwright • Docker

3. How Everyone's Work Connects



4. Shared API Contracts

Module	Endpoint	Output
Member 3 — Search	POST /search/candidates	Top-K potentially conflicting registered titles.
Member 2 — ML	POST /ml/similarity	Lexical, phonetic, semantic and core-word scores.
Member 4 — Rules	POST /rules/check	Triggered rules, severity, messages and references.
Member 1 — Verification	POST /verify-title	Final combined verdict and evidence.
Member 5 — Generation	POST /generate-titles	Generated, verified and ranked alternatives.

5. Recommended Build Order

1. First: Member 1 and Member 6 create the basic backend/frontend skeleton.
2. Second: Member 3 loads the title dataset and exposes candidate search.
3. Third: Member 2 builds the similarity engine.
4. Fourth: Member 4 builds the deterministic rules and explanation layer.
5. Fifth: Member 1 integrates all three verification components.
6. Sixth: Member 6 connects the real frontend to the integrated API.
7. Finally: Member 5 adds title generation and ranking on top of the working verification engine.

6. MVP Priority

Priority	Features
Must Have	Title input • language handling • normalization • candidate retrieval • lexical/phonetic/semantic similarity • core-word matching • deterministic rules • verdict • explanation
Next	Guideline-grounded RAG • better visualization • analytics
If Time Permits	Generator → Verifier → Ranker • officer-style assistant • advanced analytics

7. One-Sentence Role Summary

- Member 1: Makes everyone's modules work together through the backend.
- Member 2: Determines how similar two titles are using NLP/ML.
- Member 3: Finds the most likely conflicting titles quickly from the database.
- Member 4: Checks the official rules and explains why a title passed or failed.
- Member 5: Generates alternative titles and keeps only the ones that pass verification.
- Member 6: Builds what the user sees and makes the complete system demo-ready.

ONE TEAM • SIX OWNERS • THREE LAYERS • ONE VERIFICATION PIPELINE